Assignment-16

Five Tools for Proactive Secure Software Development

A proactive security posture shifts vulnerability detection left in the software development life cycle (SDLC), identifying and remediating defects before deployment rather than responding to exploitation in production. The following industry-standard tools support this approach.

1. **Fortify Static Code Analyzer (SAST).** Fortify, now part of OpenText, performs static application security testing by parsing source code and analyzing data flow, control flow, and taint propagation to detect vulnerabilities such as SQL injection, buffer overflows, and insecure deserialization. It integrates into CI/CD pipelines to enforce security gates at build time.
2. **Burp Suite (DAST and Manual Testing).** Burp Suite, from PortSwigger, functions as an intercepting proxy between the client and the target application, allowing testers to capture, modify, and replay HTTP/S traffic. Its scanner and tooling identify runtime flaws such as cross-site scripting (XSS), cross-site request forgery (CSRF), and broken authentication or session management.
3. **OWASP Dependency-Check (SCA).** Dependency-Check performs software composition analysis by fingerprinting third-party libraries and mapping them to Common Platform Enumeration (CPE) identifiers, then correlating them against the National Vulnerability Database for known CVEs. This mitigates supply-chain risk from vulnerable or outdated components.
4. **Nessus (Vulnerability Assessment).** Nessus, from Tenable, conducts credentialed and non-credentialed scans of hosts and network services to detect missing patches, misconfigurations, and known vulnerabilities scored by CVSS. It validates the security baseline of the infrastructure hosting the application.
5. **Metasploit Framework (Penetration Testing).** Metasploit, from Rapid7, provides a modular framework of exploits, payloads, and post-exploitation modules for controlled adversary emulation. It verifies whether identified vulnerabilities are actually exploitable, supporting risk-based remediation prioritization.

Importance of Security Planning

Security planning embeds security requirements, threat modeling, secure design review, and verification activities into each SDLC phase rather than treating security as a post-

development activity. Remediation cost increases significantly as defects progress from design to production, where fixes may require emergency patching, service disruption, and incident response (Richardson & Thies, 2013).

A formal security plan defines security requirements, assigns ownership, establishes risk acceptance criteria, and specifies testing and validation controls. This reduces the organization's attack surface, protects the confidentiality, integrity, and availability (CIA) of data and systems, and supports compliance with regulatory and industry frameworks. Without structured planning, security assurance is inconsistent, and vulnerabilities are often discovered only after exploitation.

Conclusion

SAST, DAST, SCA, vulnerability assessment, and penetration testing tools enable early detection and validation of security defects. Combined with security planning across the SDLC, they reduce risk exposure and lower remediation cost.

Reference

Richardson, T., & Thies, C. N. (2013). *Secure software design*. Jones & Bartlett Learning.